CS 5402: Introduction to Data Mining

SP2026

(Lecture -8)

Perceptron

Dr. Huiyuan Yang

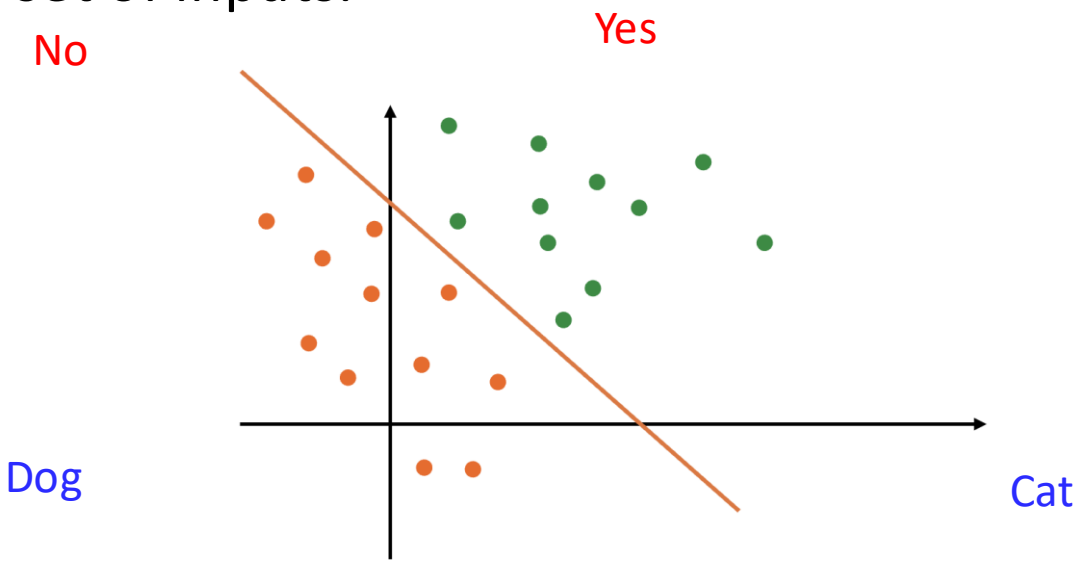
Department of Computer Science
Missouri S&T

https://hyang428.github.io

MISSOURI S&T

What is Perceptron?

- The oldest and simplest “artificial neuron” model (Frank Rosenblatt, 1957)
- It’s a **linear binary** classifier.
- It’s an algorithm that learns to make a decision (Yes/No) based on a set of inputs.



Perceptron

- Given data $D = \{(x_i, y_i)\}_{i=1}^{N}$, $x_i \in \mathcal{R}^d$, $\mathrm{y_i} \in \{+1, -1\}$, and we often assume that data D is linearly separable.
- A *linear discriminant function* is defined as:

$$f_{w,b}(x) = w^T x + b$$

- w is a weight vector,
- x is input
- b is bias

- If we have:
  - $w^T x + b > 0$ for all +1 class
  - $w^T x + b < 0$ for all -1 class

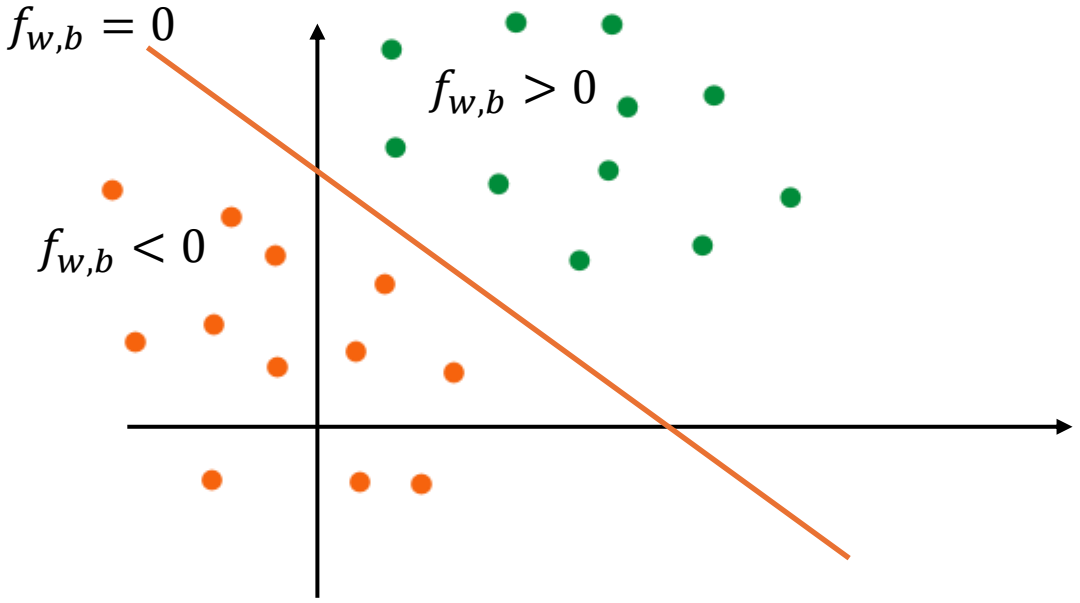
Goal: Correctly classify all the samples

- Then we call such a linear binary classifier as *Perceptron*.

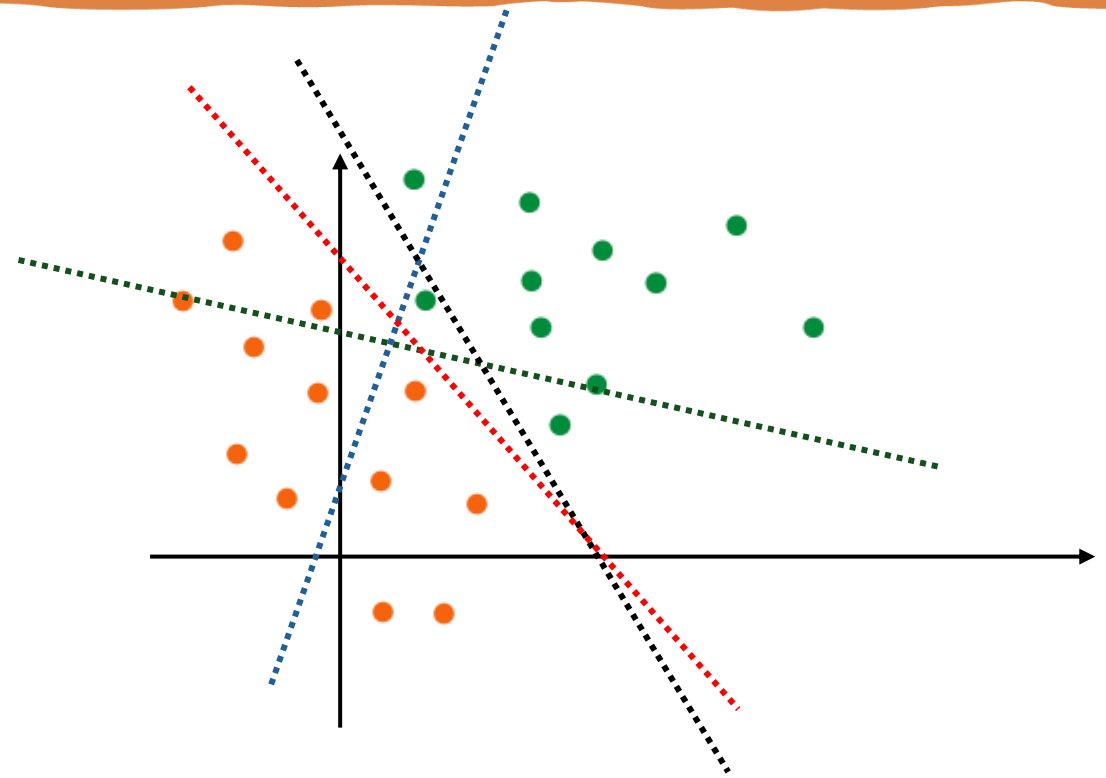
Perceptron Example

- Illustration of the 2D case

$$f_{w,b}(x) = w^T x + b, \qquad x_i = \left[x_{i,1}, x_{i,2}\right]^T$$



Perceptron – Intuition

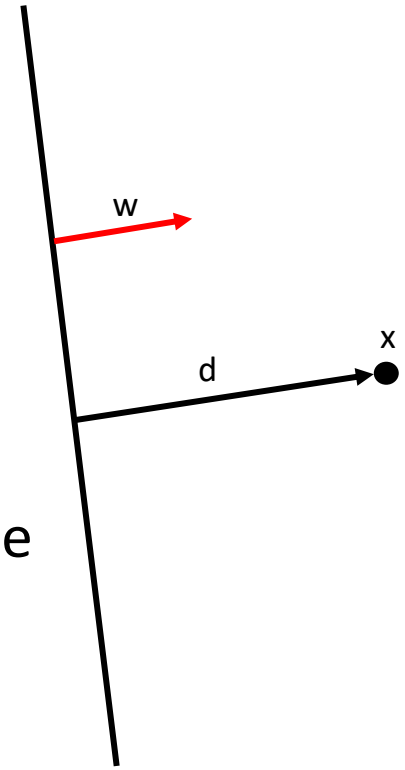


Which line is the best fit for this classification task?

Review: Margin

Given hyperplane:

$$f_{w,b}(x) = w^T x + b = 0$$

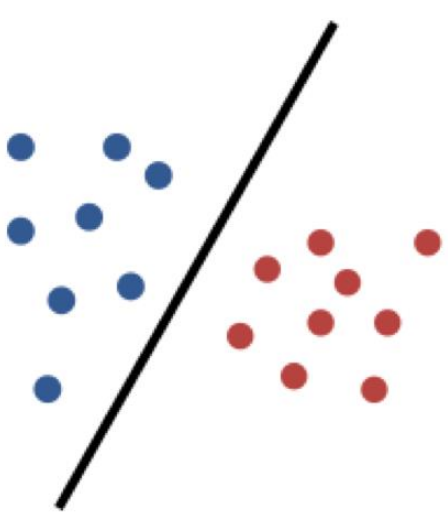


- Claim 1: w is orthogonal to the hyperplane

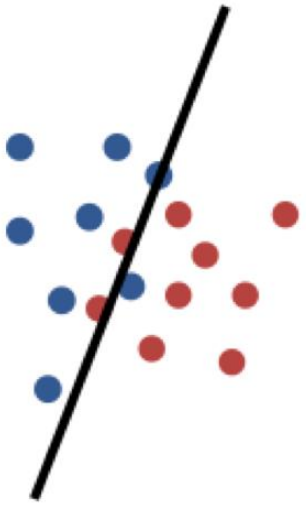
- Claim 2: for any x, the distance to the hyperplane

$$d = \frac{|f_{w,b}(x)|}{||w||}$$

Review: Linearly Separable



Linearly separable



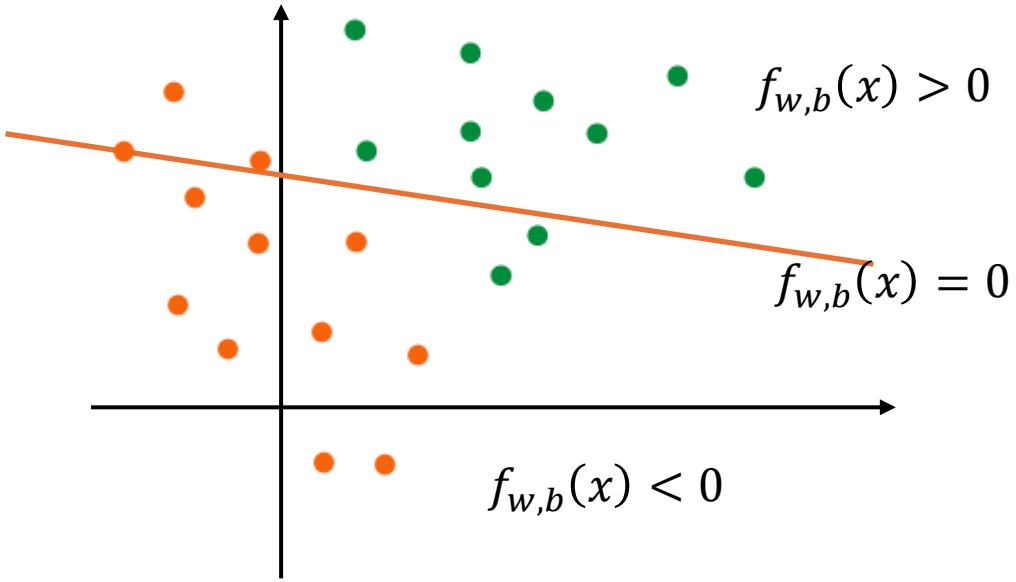
Not linearly separable

Loss Function (1)

Objective: learn a hyperplane $f_{w,b}(x) = w^T x + b$ that **correctly separate** all the samples in dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, with:

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

Define a loss function!!



Loss Function (2)

Objective: learn a hyperplane $f_{w,b}(x) = w^T x + b$ that correctly separate all the samples in dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, with:

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

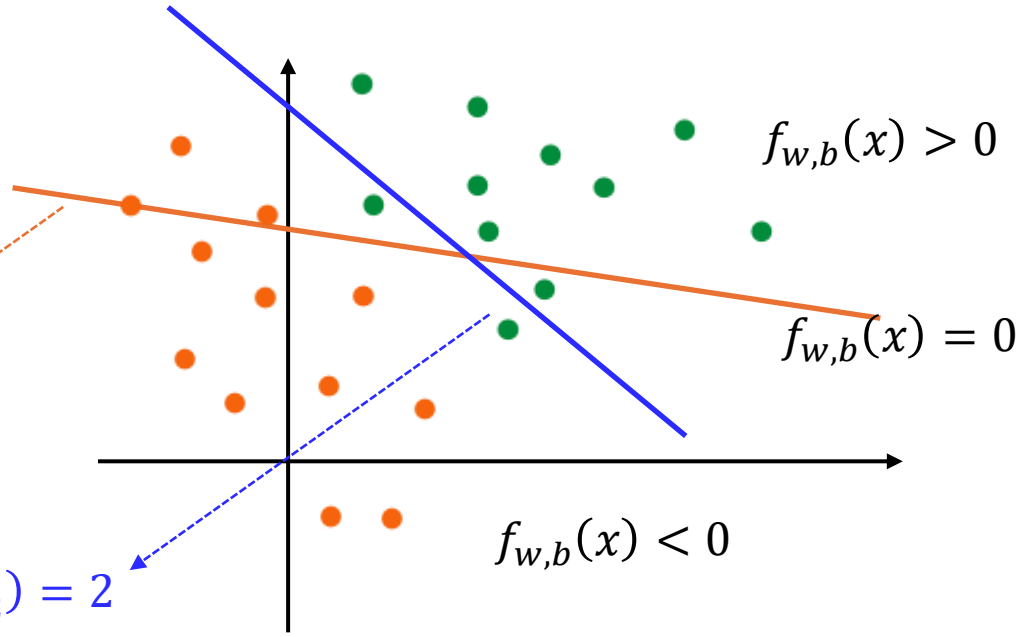
Define a loss function!!

- 1st try: Count the number of misclassified samples

$L(w_1, b_1) = 4$

Not differentiable!

$L(w_2, b_2) = 2$



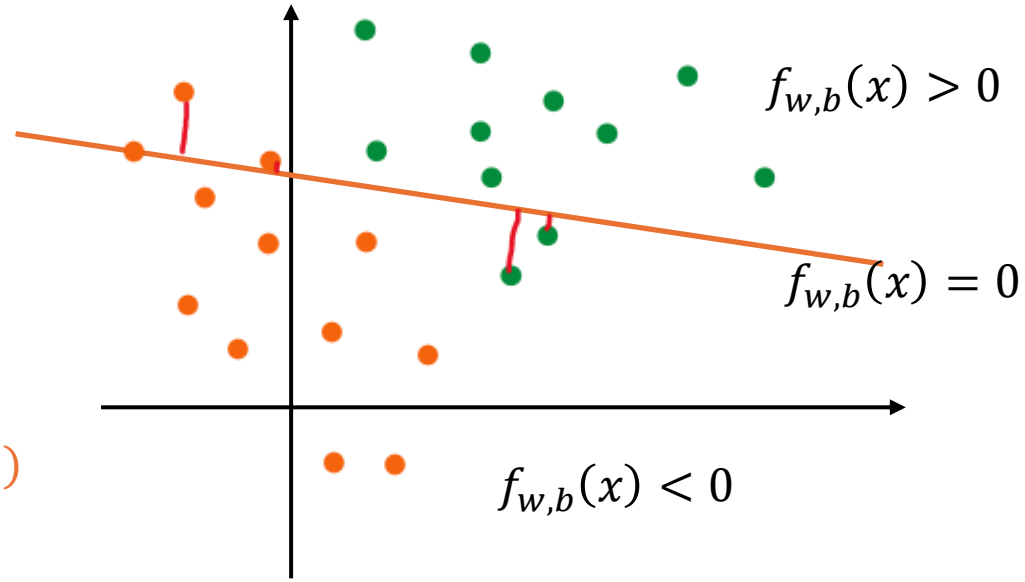
Loss Function (3)

Objective: learn a hyperplane $f_{w,b}(x) = w^T x + b$ that correctly separate all the samples in dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, with:

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

Define a loss function!!

- 2nd try: Count the distances of misclassified samples



$$L(w_1, b_1) = \sum_{x_i \in M} dist(x_i, f_{w,b})$$

If a point is misclassified: adjust weights to reduce the error

Loss Function (4)

Correct classification

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

$$y_i(w^T x_i + b) > 0$$

Misclassification

- $w^T x + b < 0$ for all +1 class
- $w^T x + b > 0$ for all -1 class

$$y_i(w^T x_i + b) < 0$$

$$y_i \in \{+1, -1\}$$

Loss Function (5)

Correct classification

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

Misclassification

- $w^T x + b < 0$ for all +1 class
- $w^T x + b > 0$ for all -1 class

$$y_i(w^T x_i + b) > 0$$

$$y_i(w^T x_i + b) < 0$$

The distance from any point x_i to hyperplane is defined as:

$$d = \frac{|w^T x_i + b|}{||w||}$$

$||w||$ is the L_2 norm

Loss Function (6)

Correct classification

- $w^T x + b > 0$ for all +1 class
- $w^T x + b < 0$ for all -1 class

Misclassification

- $w^T x + b < 0$ for all +1 class
- $w^T x + b > 0$ for all -1 class

$$y_i(w^T x_i + b) > 0$$

$$y_i(w^T x_i + b) < 0$$

Further, assume M: the set of misclassified samples

The distance from any point $x_i \in M$ to hyperplane is defined as:

$$d = \frac{|w^T x_i + b|}{||w||} = \frac{-y_i(w^T x_i + b)}{||w||}$$

$||w||$ is the L_2 norm

Loss Function (7)

Loss function is then defined as:

$$L(w, b) = -\frac{1}{||w||} \sum_{x_i \in M} y_i (w^T x_i + b)$$

Where, M: the set of misclassified samples

Loss Function (8)

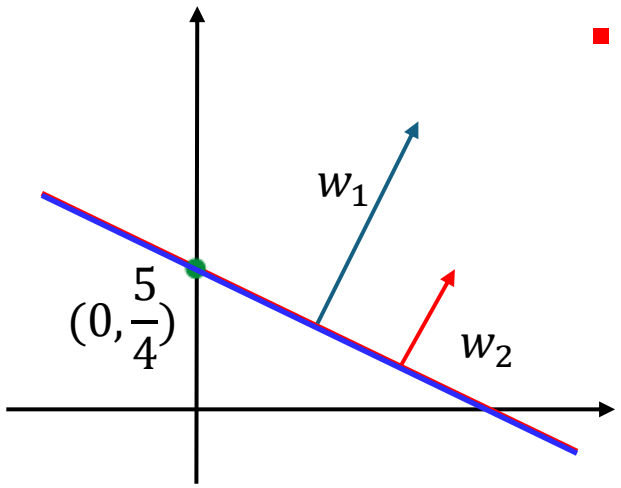
$$f_{w,b}(x) = w^T x + b = 0$$

$w_1 = [3,4]^T$

- $3x + 4y - 5 = 0$

$w_2 = \left[\frac{3}{5}, \frac{4}{5}\right]^T$

- $\frac{3}{5}x + \frac{4}{5}y - \frac{5}{5} = 0$



Therefore, we can set $||w|| = 1$, still same hyperplane.

Loss Function (9)

Find the parameter $\{w, b\}$, so that the objective function is minimized:

$$L(w,b) = -\frac{1}{||w||}\sum_{x_i \in M} y_i(w^T x_i + b)$$

We can set $||w|| = 1, then$

$$\min_{w,b} L(w,b) = -\sum_{x_i \in M} y_i(w^T x_i + b)$$

M: the set of misclassified samples

Gradient Descent!

What is Gradient Descent?

Optimization Problem

- Given loss function:

$$L(\mathbf{w})$$

We want to:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} L(\mathbf{w})$$

- **Core Idea:** move parameters in the **negative gradient direction**:

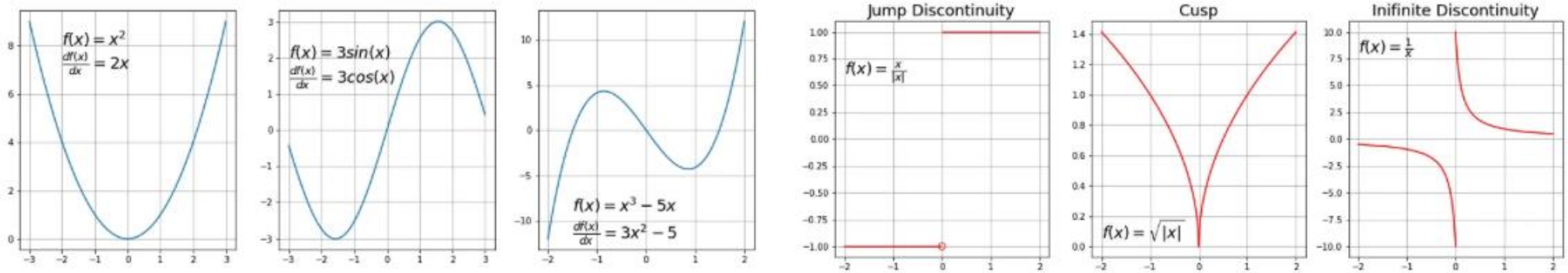
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$$

Where:

- $\nabla L(\mathbf{w}) =$ gradient
- $\eta =$ learning rate

Basics: Gradient Descent

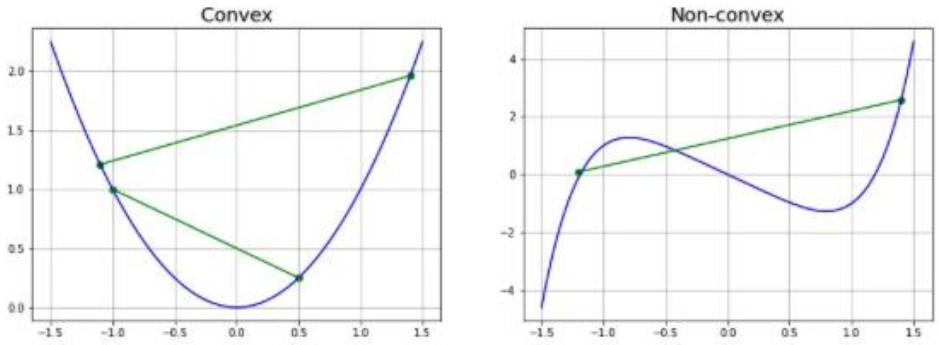
- Gradient descent (GD) is an iterative first-order optimization algorithm, used to find a local minimum/maximum of a given function.
- GD does not work for all functions, and there are two specific requirements:
  - Differentiable



https://towardsdatascience.com/gradient-descent-algorithm-a-deep-dive-cf04e8115f21

Basics: Gradient Descent (Cont'd)

- Gradient descent (GD) is an iterative first-order optimization algorithm, used to find a local minimum/maximum of a given function.
- GD does not work for all functions, and there are two specific requirements:
  - Differentiable
  - convex



https://towardsdatascience.com/gradient-descent-algorithm-a-deep-dive-cf04e8115f21

An Intuitive Analogy:



Finding the Bottom of a Valley in the Fog

The Hiker's Strategy

A Step-by-Step Approach

You'd probably follow a simple algorithm:

- **Look around your feet** to feel which direction is the steepest **downhill**.
- **Take a small step** in that direction.
- **Repeat** this process until you can't go any further downhill.

This simple, iterative strategy is **EXACTLY** the intuition behind Gradient Descent.

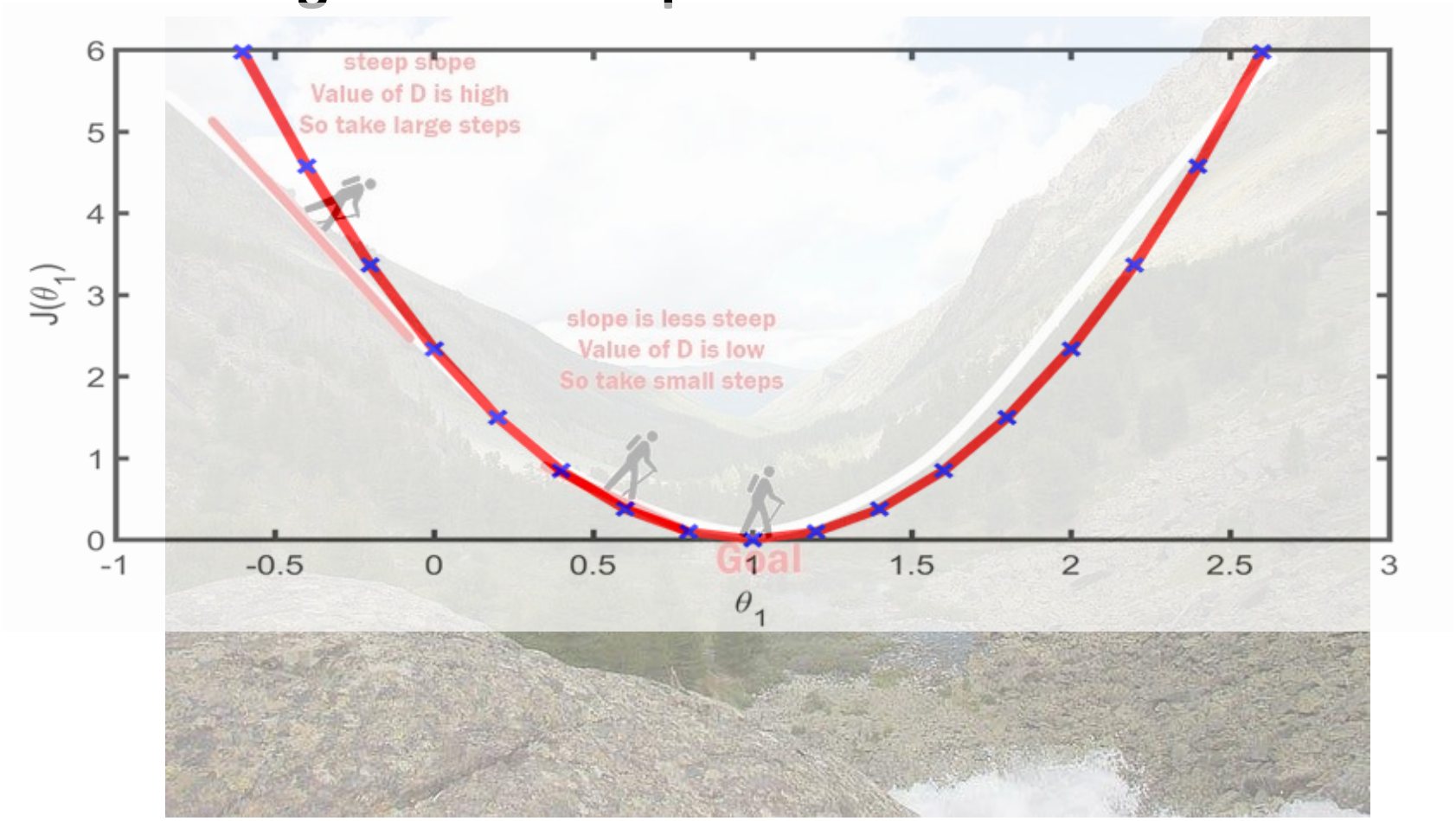
Connecting the Analogy to Our Problem

Translating Hiking to Machine Learning

Hiker's Analogy	Machine Learning Concept
Your position on the mountain (longitude, latitude)	The **parameters** of our model (the slope m and intercept b)
The altitude of the ground	The **Cost Function** Cost(m, b)
The lowest point in the valley	The **minimum cost** (our goal!)
The direction of steepest descent	The **Gradient** (the derivative of the cost function)
The size of your step	The **Learning Rate**
Taking a step	Updating the parameters m and b

Ingredient 1: The Cost Function (The Valley)

Visualizing the Landscape:



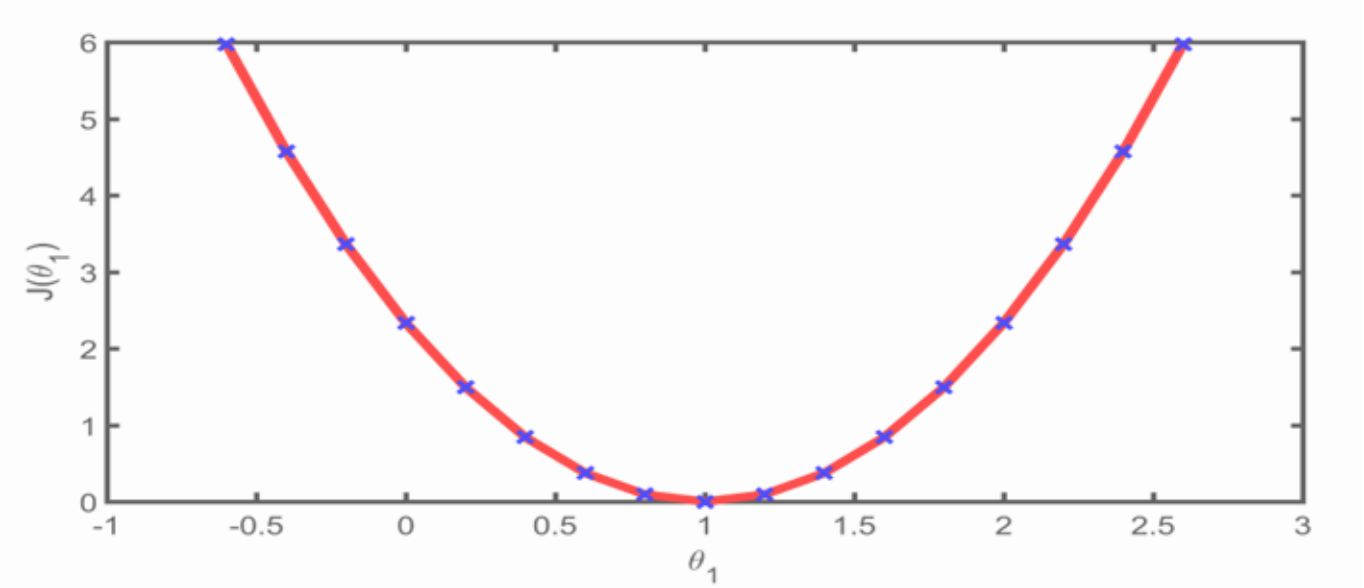
Ingredient 2: The Gradient (Which Way is Down?)

How do we know which way is the steepest downhill?

The **derivative** of a function at a point tells us the slope of the tangent line at that point. The **gradient** is just a multi-dimensional version of the derivative.

- If the slope is **negative**, we need to move **right** (increase the parameter).
- If the slope is **positive**, we need to move **left** (decrease the parameter).

The gradient always points in the direction of the **steepest ASCENT** (uphill). So, we just take a step in the **OPPOSITE** direction.



Ingredient 3: The Learning Rate (How Big a Step?)

Taking a Step in the Right Direction

Okay, we know the direction. But how far should we go with each step?

This is controlled by a setting we choose called the **Learning Rate** (often denoted by alpha, α, or eta, η).

- It's a small positive number (e.g., 0.01, 0.001).
- It determines how much we update our parameters (m and b) in each iteration.

Step Size = Learning Rate × Gradient

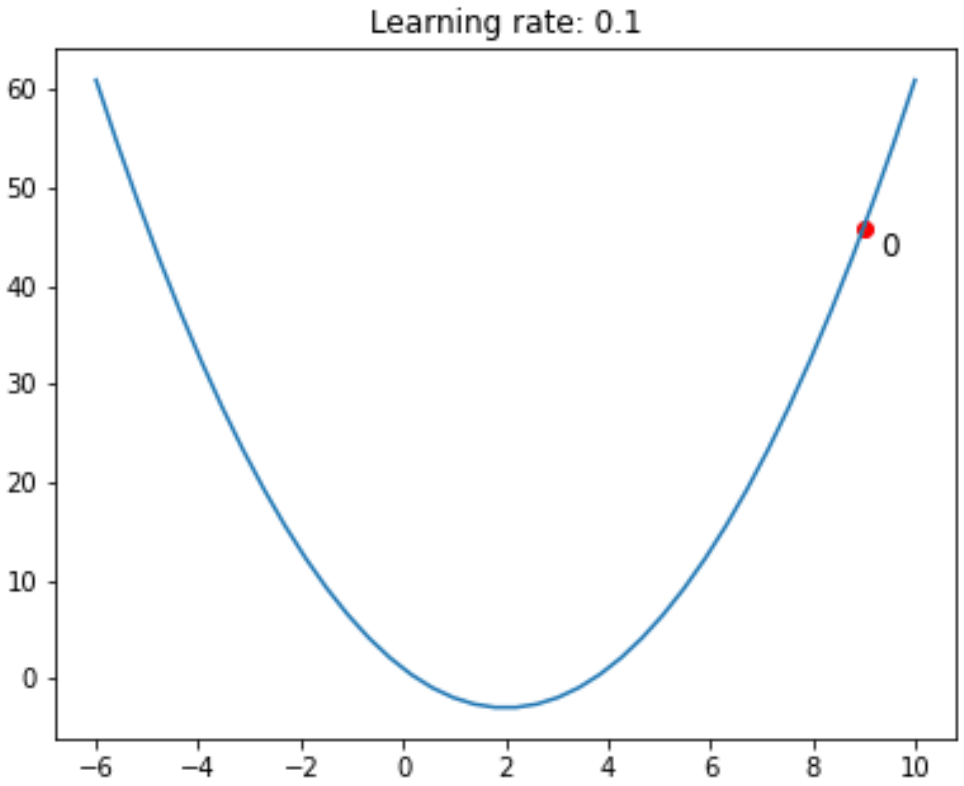
Choosing a good learning rate is critical, as we'll see later. For now, think of it as the size of the hiker's stride.

The Gradient Descent Algorithm

Putting It All Together

1. **Initialize:** Start with random values for your parameters m and b. (This is like dropping the hiker randomly onto the mountain).
2. **Calculate Gradient:** Compute the gradient of the Cost Function with respect to m and b at your current position. (This is figuring out the steepest downhill direction).
3. **Update Parameters:** Take a small step in the opposite direction of the gradient.
   - new_θ = current_ θ - learning_rate * gradient_ θ
4. **Repeat:** Go back to Step 2 and repeat for a set number of iterations, or until the cost stops changing much.

Visualizing The Process



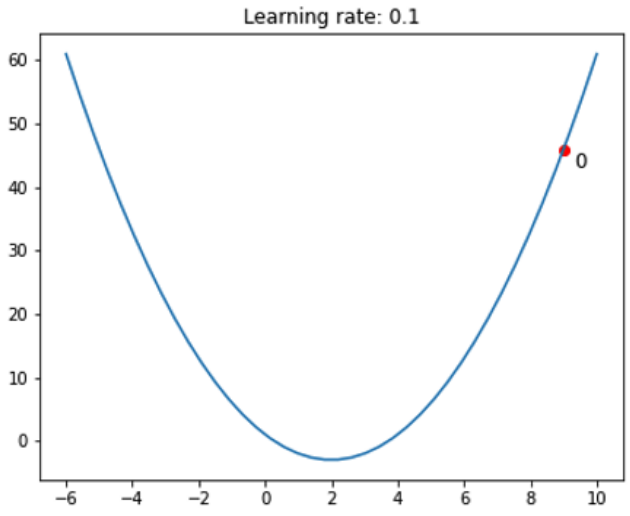
The algorithm iteratively adjusts θ to find the values that minimize our error. It's literally "descending" the "gradient".

A Simple Numerical Example (Part 1)

- Example: find the minimal value for the function defined as:

$$f(x) = x^2 - 4x + 1$$

$$x_{i+1} = x_i - \alpha \frac{\partial f}{\partial x}(x_i) \quad \text{and gradient:} \quad \frac{\partial f(x)}{\partial x} = 2x - 4$$



Initial: $x_0 = 9$.

$$\rightarrow x_1 = x_0 - \alpha \cdot \frac{\partial f}{\partial x}(x_0) = 9 - 0.2 \times (2 \times 9 - 4) = 6.2$$

$$\rightarrow x_2 = x_1 - \alpha \cdot \frac{\partial f}{\partial x}(x_1) = 6.2 - 0.2 \times (2 \times 6.2 - 4) = 4.52$$

$$\rightarrow x_3 = x_2 - \alpha \cdot \frac{\partial f}{\partial x}(x_2) = 4.52 - 0.2 \times (2 \times 4.52 - 4) = 3.512$$

$$\rightarrow x_4 = x_3 - \alpha \cdot \frac{\partial f}{\partial x}(x_3) = 3.512 - 0.2 \times (2 \times 3.512 - 4) = 2.91$$

$$\rightarrow x_5 = x_4 - \alpha \cdot \frac{\partial f}{\partial x}(x_4) = 2.91 - 0.2 \times (2 \times 2.91 - 4) = 2.546$$

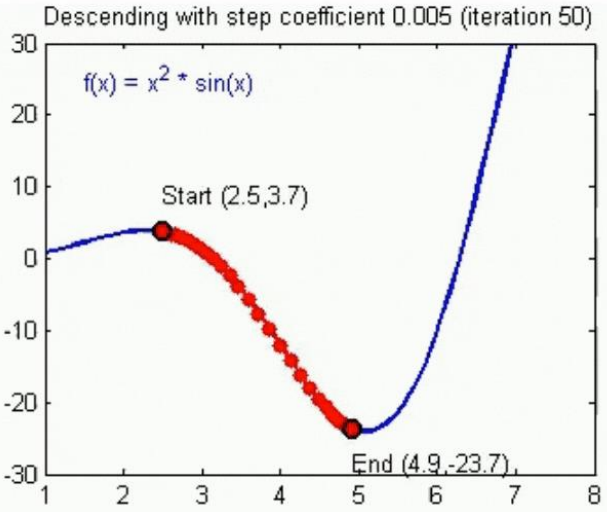
$$\rightarrow x_6 = x_5 - \alpha \cdot \frac{\partial f}{\partial x}(x_5) = 2.546 - 0.2 \times (2 \times 2.546 - 4) = 2.328$$

$$\rightarrow x_7 = x_6 - \alpha \cdot \frac{\partial f}{\partial x}(x_6) = 2.328 - 0.2 \times (2 \times 2.328 - 4) = 2.197$$

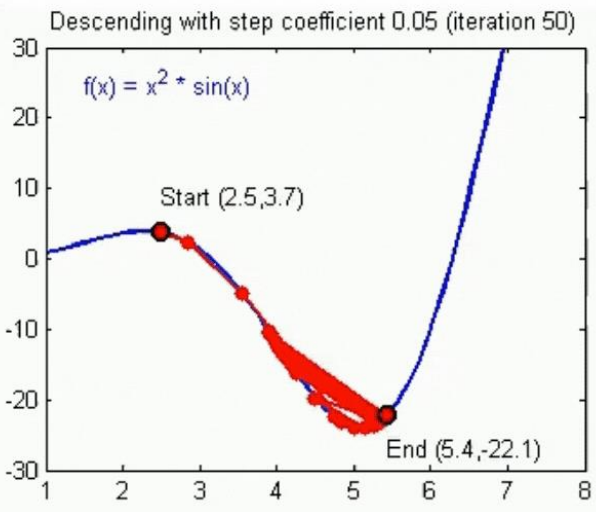
The Importance of the Learning Rate

The Learning Rate (α) is a hyperparameter that you must choose. Its value has a huge impact.

Convergence

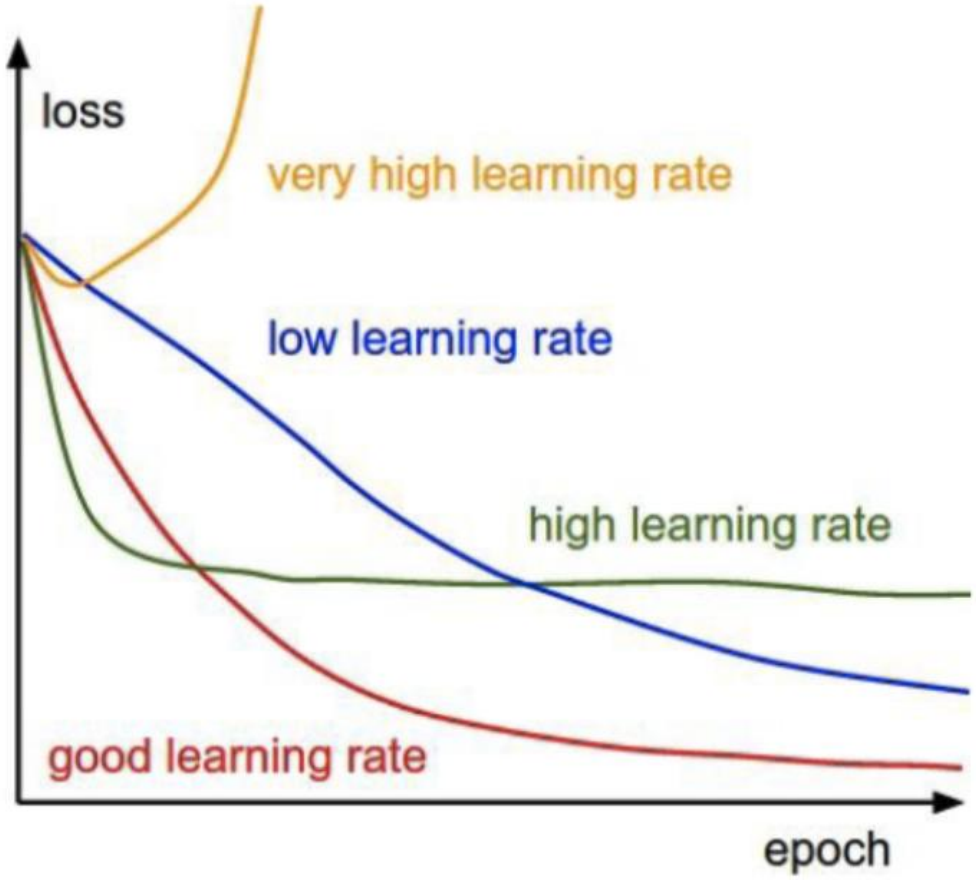


Divergence



Too Small	Just Right	Too Large
Takes tiny, inefficient steps.	Converges efficiently.	Overshoots the minimum and can **diverge**!
Result: Very slow training.	**Result:** Good performance.	**Result:** The model breaks.

Effect of Learning rate on the loss function



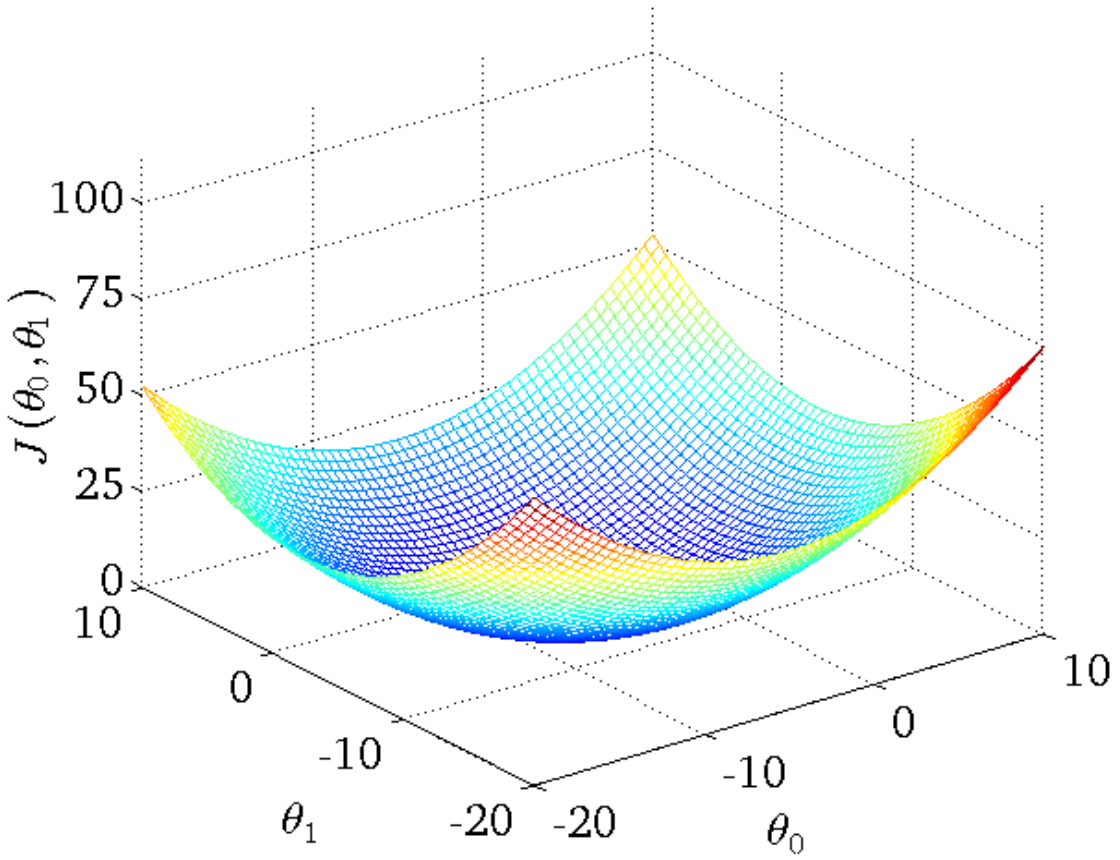
https://srdas.github.io/DLBook/GradientDescentTechniques.html

Types of Gradient Descent

Two common variations are:

- **Stochastic Gradient Descent (SGD):** Updates the parameters using the gradient from just **one** random data point at a time. It's noisy but fast.
- **(Mini-) Batch Gradient Descent:** A compromise. Updates the parameters using a small batch (e.g., 32 or 64) of data points. This is the most common method used in practice today.

Gradient descent with convex function



"convex function"

Local minimum
= global minimum

Gradient descent for general cost functions (1)

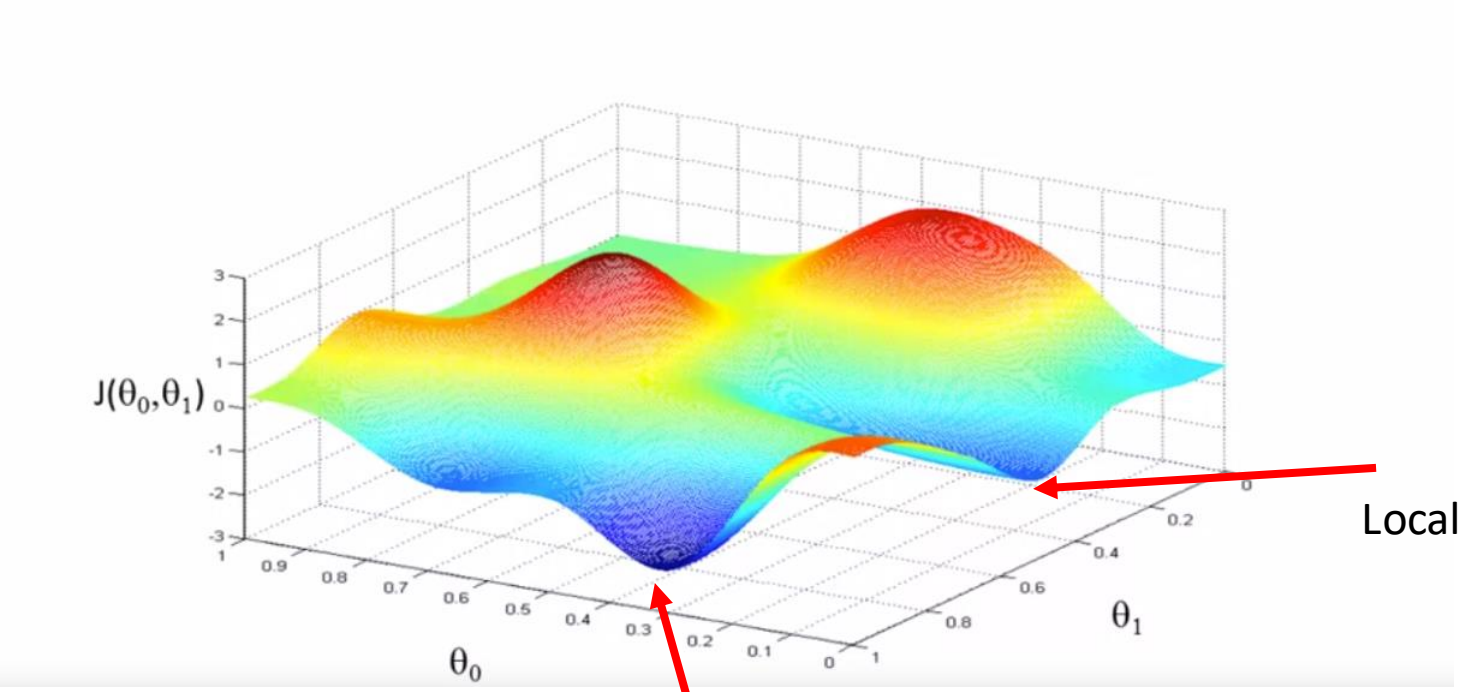


Image courtesy Dr. Andrew Ng, Stanford University Machine learning

Gradient descent for general cost functions (2)

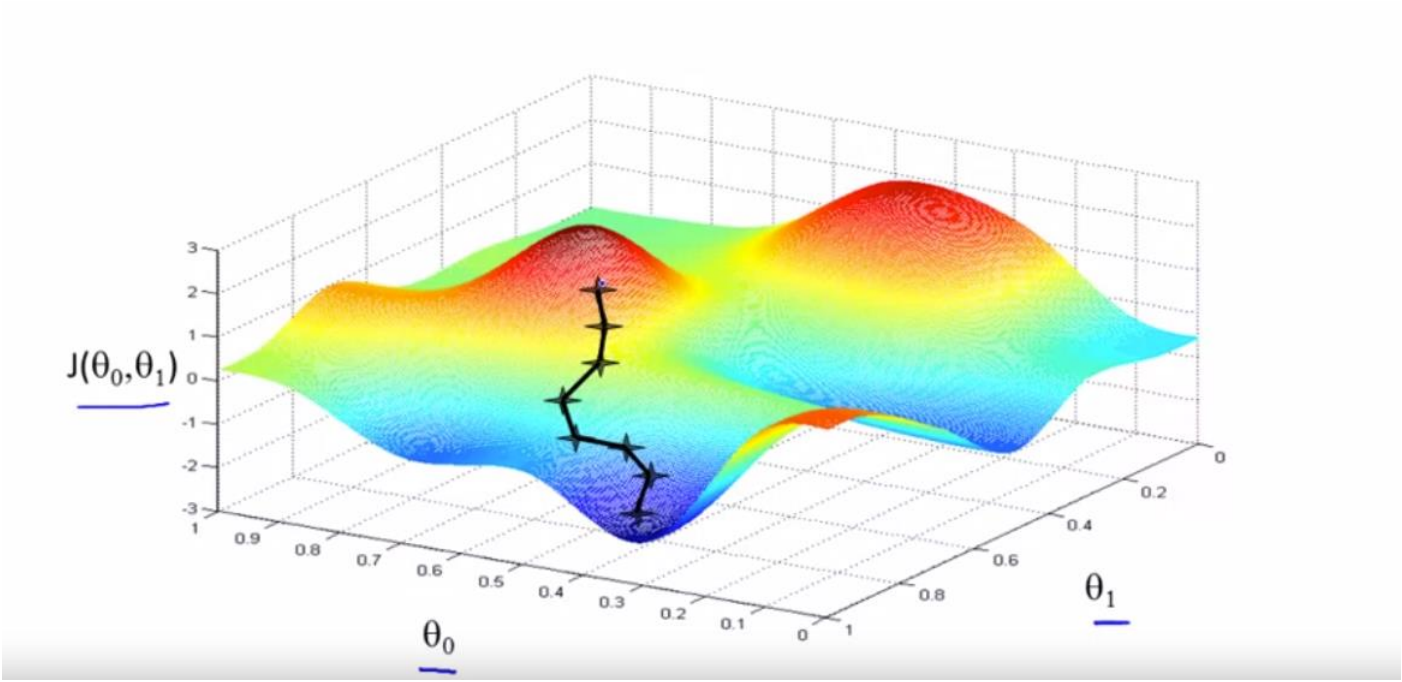


Image courtesy Dr. Andrew Ng, Stanford University Machine learning

Gradient descent for general cost functions (3)

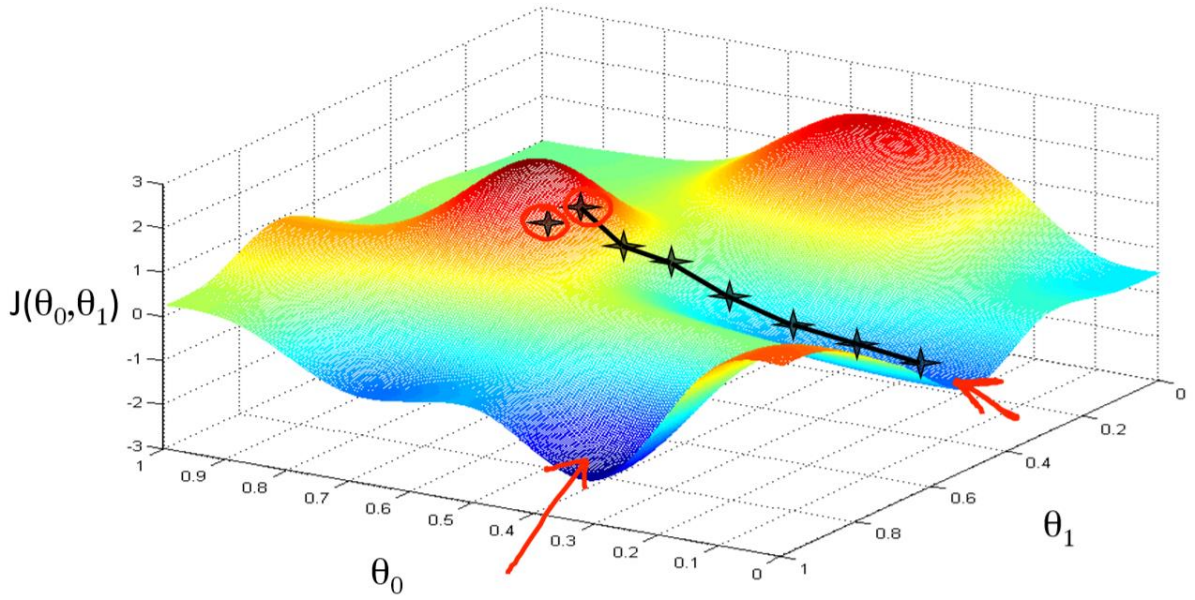


Image courtesy Dr. Andrew Ng, Stanford University Machine learning

Perceptron Algorithm with SGD

- Stochastic Gradient Descent (SGD) algorithm is used the solve the optimization problem.

$$\min_{w,b} L(w,b) = -\sum_{x_i \in M} y_i (w^T x_i + b)$$

- Gradient is then calculated by:

$$\nabla_w L(w,b) = -\sum_{x_i \in M} y_i x_i \qquad \nabla_b L(w,b) = -\sum_{x_i \in M} y_i$$

- Randomly pick a misclassified sample (x_i, y_i), update w, b:

$$w \leftarrow w - \eta \nabla_w L(w,b) = w + \eta y_i x_i$$

$$b \leftarrow b - \eta \nabla_b L(w,b) = w + \eta y_i$$

η is called learning rate.

Perceptron Learning Algorithm

Input: $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, $x_i \in R^d$, $y_i \in \{+1, -1\}$, and learning rate η $(0 < \eta \leq 1)$

Output: w, b

---

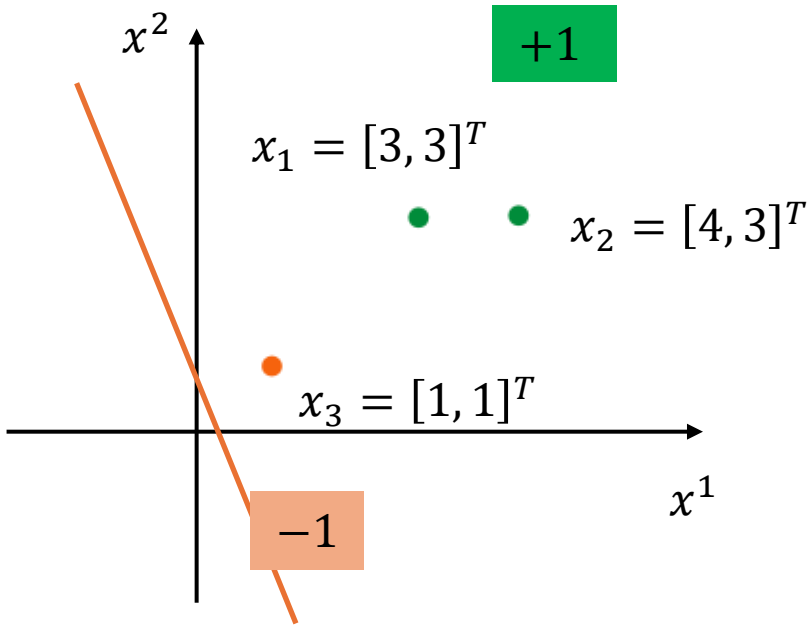
1) Initialize the weight vector w_0 and bias b_0
2) Pick a sample (x_i, y_i) from the dataset D
   - If x_i is correctly classified, do nothing
   - Else: update the parameters with:

$$w \leftarrow w + \eta y_i x_i$$

$$b \leftarrow b + \eta y_i$$

3) repeat step-2 until all the samples are correctly classified.

Example-1: (1)



Find w, b with $\eta = 1$

step-1. Initialization:

$$w_0 = [2,1]^T,\ b_0 = -\frac{1}{2}$$

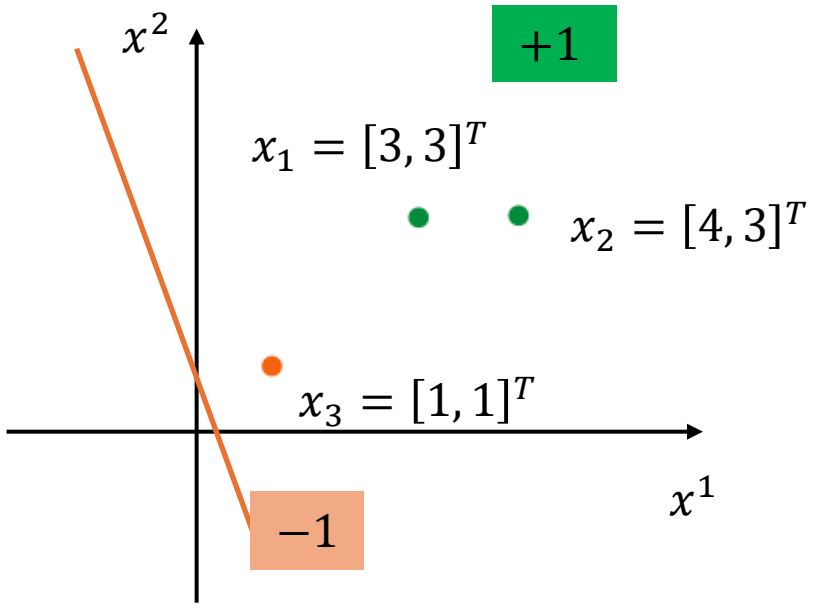
$$f_{w,b}(x) = [2,1]\begin{bmatrix} x^1 \\ x^2 \end{bmatrix} - \frac{1}{2}$$

$f_{w,b}(x_1) > 0$ +1

$f_{w,b}(x_2) > 0$ +1

$f_{w,b}(x_3) > 0$ −1

Example-1: (2)



Find w, b with $\eta = 1$

step-1. Initialization:

$$w_0 = [2,1]^T,\ b_0 = -\frac{1}{2}$$

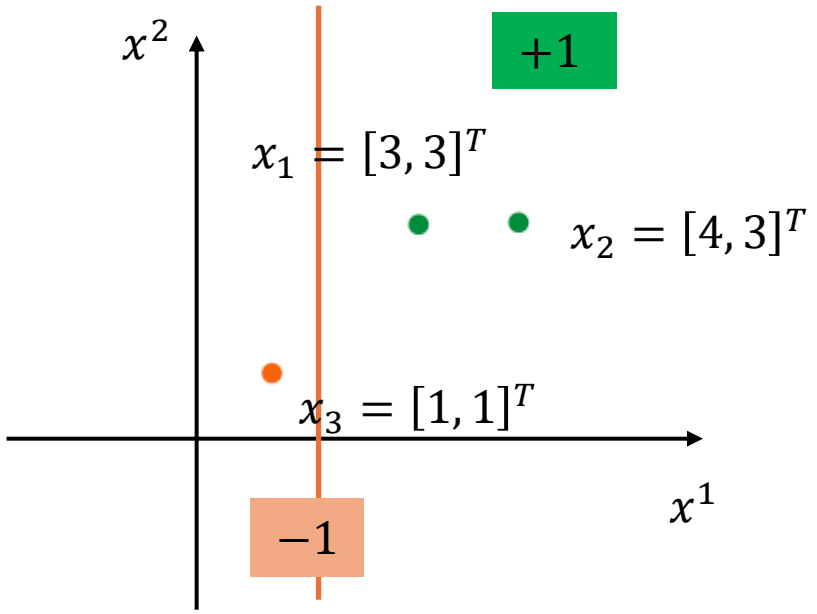
$$f_{w,b}(x) = [2,1]\begin{bmatrix} x^1 \\ x^2 \end{bmatrix} - \frac{1}{2}$$

Update w, b with misclassified (x_3, y_3)

$$w_1 = w_0 + y_3 x_3 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + (-1) \times \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$b_1 = b_0 + y_3 = -\frac{1}{2} + (-1) = -\frac{3}{2}$$

Example-1: (3)



Find w, b with $\eta = 1$

step-2.

$$w_1 = \begin{bmatrix}1\\0\end{bmatrix},\ b_1 = -\frac{3}{2}$$

$$f_{w,b}(x) = [1,0]\begin{bmatrix}x^1\\x^2\end{bmatrix} - \frac{3}{2}$$

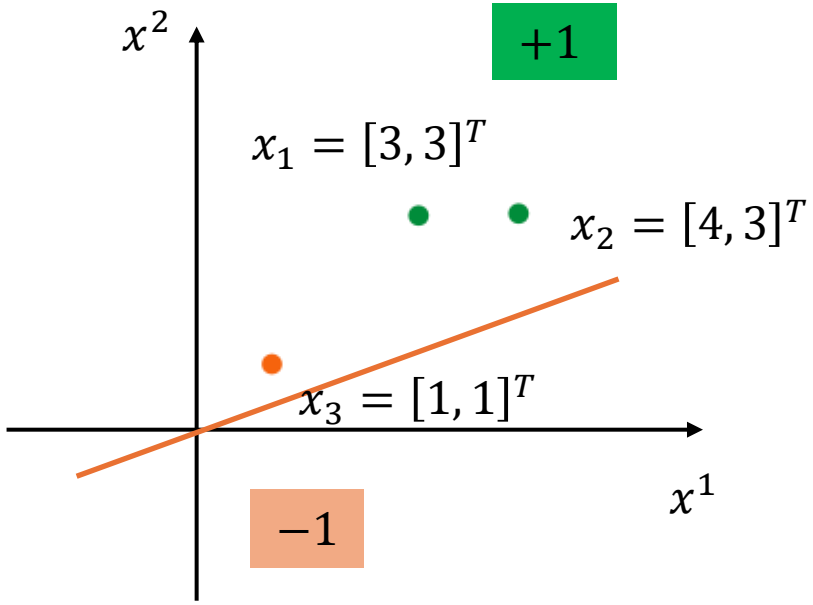
$f_{w,b}(x_1) > 0$ +1

$f_{w,b}(x_2) > 0$ +1

$f_{w,b}(x_3) < 0$ −1

Converge and stop!

Example-2: (1)



Find w, b with $\eta = 1$

step-1. Initialization:

$$w_0 = \left[\frac{1}{2}, -1\right]^T, \; b_0 = 0$$

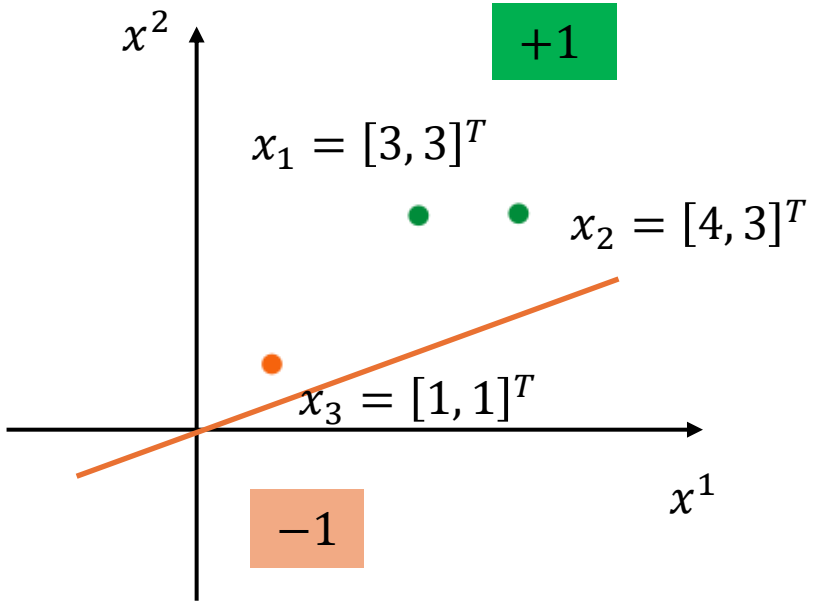
A different initialization.

Example-2: (2)

Find w, b with $\eta = 1$

step-1. Initialization:

$$w_0 = \left[\frac{1}{2}, -1\right]^T, \quad b_0 = 0$$



$h_0(x) = \frac{1}{2}x^1 - x^2 \Rightarrow$

$y_1 \cdot h_0(x_1) < 0$ ✗

$y_2 \cdot h_0(x_2) < 0$ ✗

$y_3 \cdot h_0(x_3) > 0$ ✓

Misclasified: $M = \{x_1, x_2\}$.

Randomly pick x_2:

$$w_1 = w_0 + y_2 x_2 = \begin{bmatrix}\frac{1}{2}\\ -1\end{bmatrix} + (+1)\begin{bmatrix}4\\3\end{bmatrix} = \begin{bmatrix}\frac{9}{2}\\2\end{bmatrix}$$

$$b_1 = b_0 + y_2 = 0 + (+1) = 1$$

$h_1(x) = \frac{9}{2}x^1 + 2x^2 + 1$

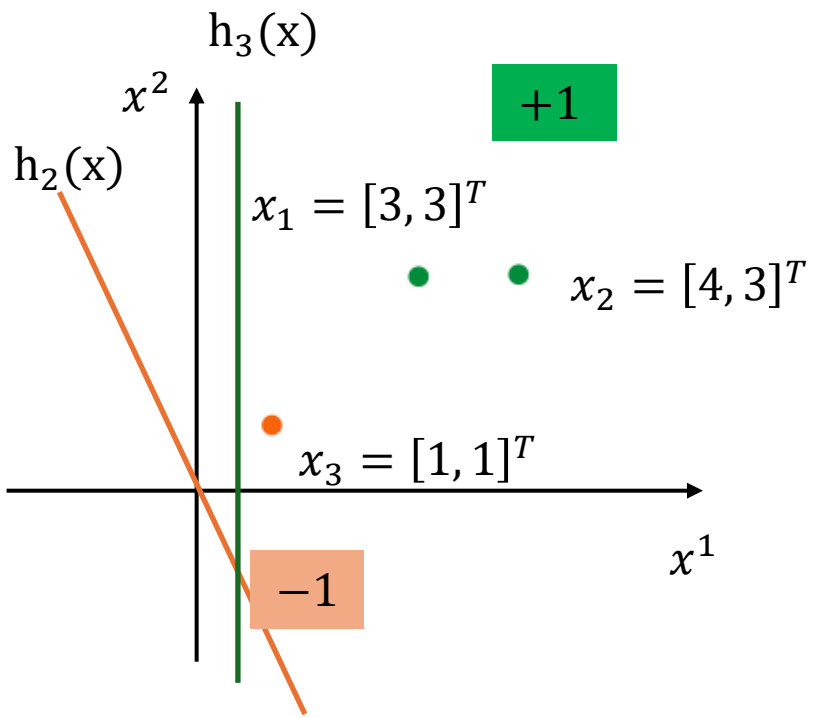
$\Rightarrow h_1(x_1) > 0$, $+1$ ✓

$h_1(x_2) > 0$ $+1$ ✓

$h_1(x_3) > 0$ -1 ✗

$M = \{x_3\}$

Example-2: (3)



$$w_2 = w_1 + y_3 \cdot x_3 = \begin{bmatrix} \frac{9}{2} \\ 2 \end{bmatrix} + (-1)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{7}{2} \\ 1 \end{bmatrix}$$

$$b_2 = b_1 + y_3 = 1 + (-1) = 0$$

$h_2(x) = \frac{7}{2}x^1 + x^2 \Rightarrow$ $y_1 \cdot h_2(x_1) > 0$ ✓

$y_2 \cdot h_2(x_2) > 0$ ✓

$y_3 \cdot h_2(x_3) < 0$ ✗

$M = \{x_3\}$

$$w_3 = w_2 + y_3 \cdot x_3 = \begin{bmatrix} \frac{7}{2} \\ 1 \end{bmatrix} + (-1)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{5}{2} \\ 0 \end{bmatrix}$$

$$b_3 = b_2 + y_3 = 0 + (-1) = -1$$

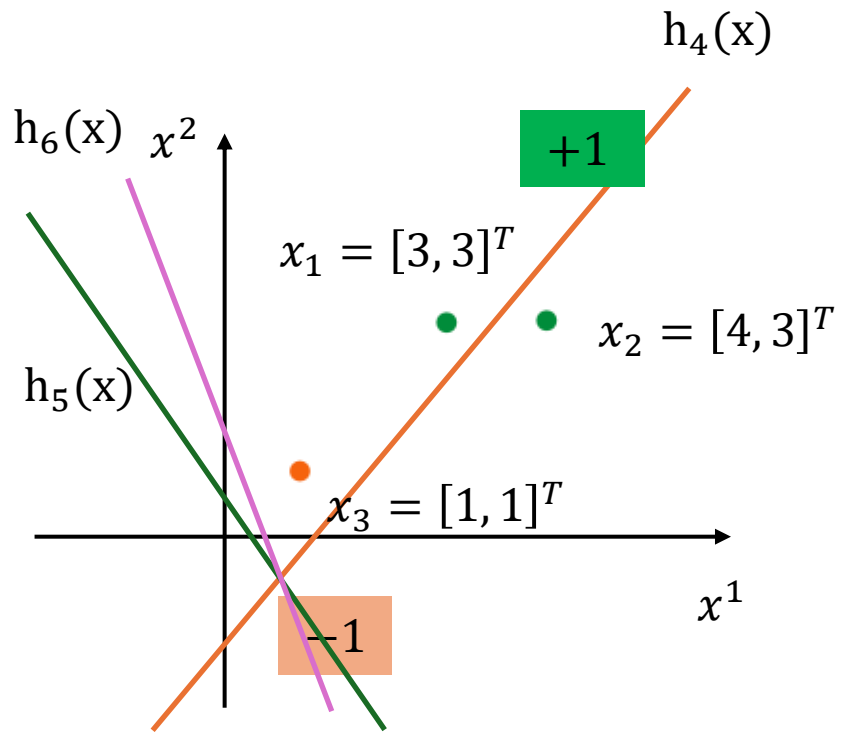
$h_3(x) = \frac{5}{2}x^1 - 1 \Rightarrow \ldots$

$M = \{x_3\}$

$$w_4 = w_3 + y_3 \cdot x_3 = \begin{bmatrix} \frac{5}{2} \\ 0 \end{bmatrix} + (-1)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{3}{2} \\ -1 \end{bmatrix}$$

$$b_4 = b_3 + y_3 = -1 + (-1) = -2$$

Example-2: (4)



$h_4(x) = \frac{3}{2}x^1 - x^2 - 2$

$M = \{x_1\}$

$w_5 = w_4 + y_1 \cdot x_1 = \begin{bmatrix} \frac{3}{2} \\ -1 \end{bmatrix} + (+1)\begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{9}{2} \\ 2 \end{bmatrix}$

$b_5 = b_4 + y_1 = -2 + (+1) = -1$

$h_5(x) = \frac{9}{2}x^1 + 2x^2 - 1$

$M = \{x_3\}$

$w_6 = w_5 + y_3 \cdot x_3 = \begin{bmatrix} \frac{9}{2} \\ 2 \end{bmatrix} + (-1)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{7}{2} \\ 1 \end{bmatrix}$

$b_6 = b_5 + y_3 = -1 + (-1) = -2$

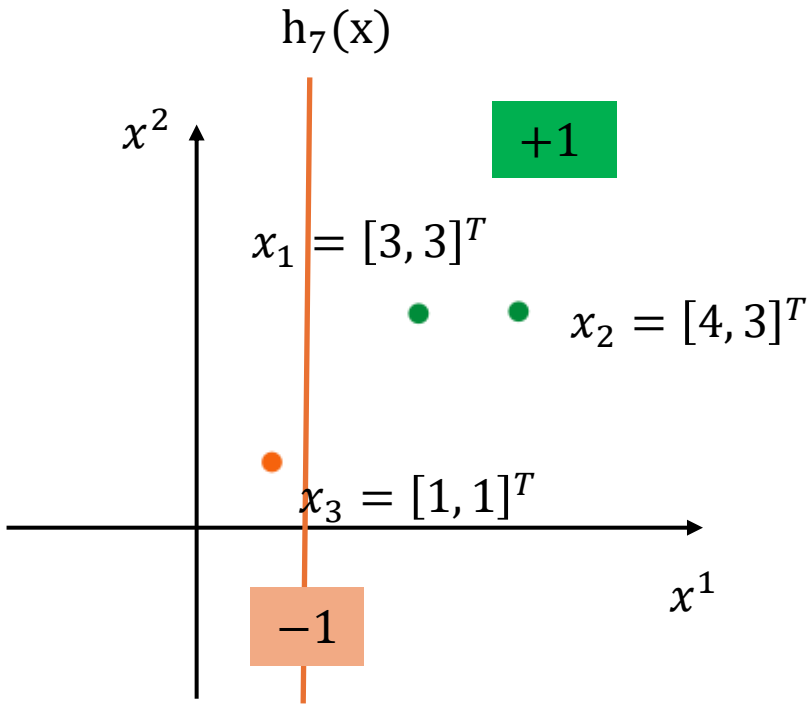
$h_6(x) = \frac{7}{2}x^1 + x^2 - 2$

$M = \{x_3\}$

$w_7 = w_6 + y_3 \cdot x_3 = \begin{bmatrix} \frac{7}{2} \\ 1 \end{bmatrix} + (-1)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{5}{2} \\ 0 \end{bmatrix}$

$b_7 = b_6 + y_3 = -2 + (-1) = -3$

Example-2: (5)



$h_7(x) = \frac{5}{2}x^1 - 3 \Rightarrow$

$y_1 \cdot h_7(x_1) > 0$ ✓

$y_2 \cdot h_7(x_2) > 0$ ✓

$y_3 \cdot h_7(x_3) > 0$ ✓



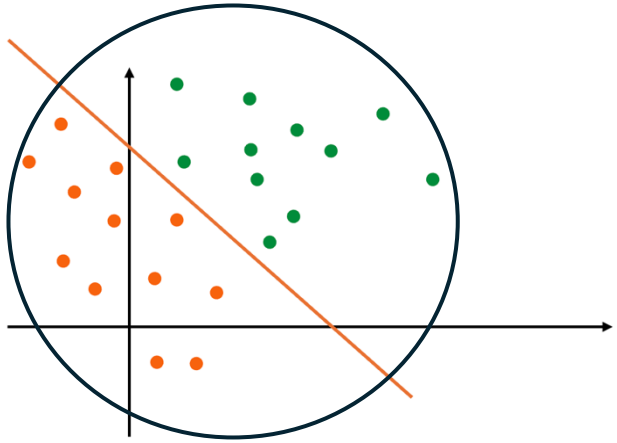
Perceptron Convergence Theorem

Assume:

- The dataset is **linearly separable**
- There exists a weight vector w^* such that

$$y_i \left(w^{*T} x_i \right) \geq \gamma > 0$$

for all training examples



- All inputs satisfy $\|x_i\| \leq R$
- Then the perceptron algorithm will Converge after **at most** $\frac{R^2}{\gamma^2}$ updates.

In-Class Participation Question (1)

If the dataset is **not linearly separable**, what happens during Perceptron training?

A. The algorithm always converges to a global minimum

B. The algorithm may keep updating indefinitely

C. The weights converge to zero

D. The algorithm switches to logistic regression

In-Class Participation Question (2)

Assume two linearly separable datasets:

- Dataset A: large margin
- Dataset B: small margin

Which statement is TRUE?

A. Perceptron converges in fewer updates on Dataset B

B. Convergence speed is independent of margin

C. Perceptron converges faster on Dataset A

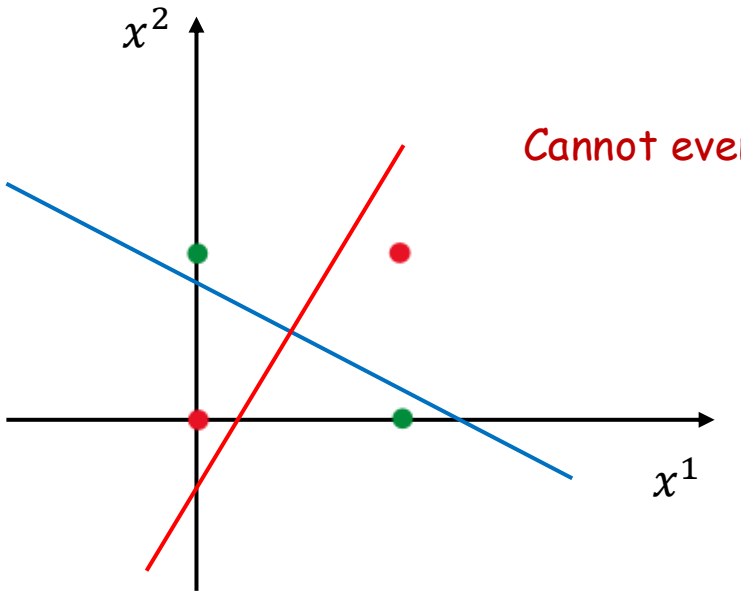
D. Perceptron always finds the maximum margin solution

Perceptron Limitation

I. Perceptron is linear model, therefore only works for linearly separable problem.

XOR problem:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

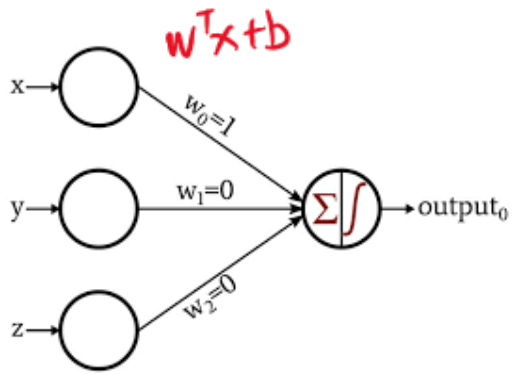


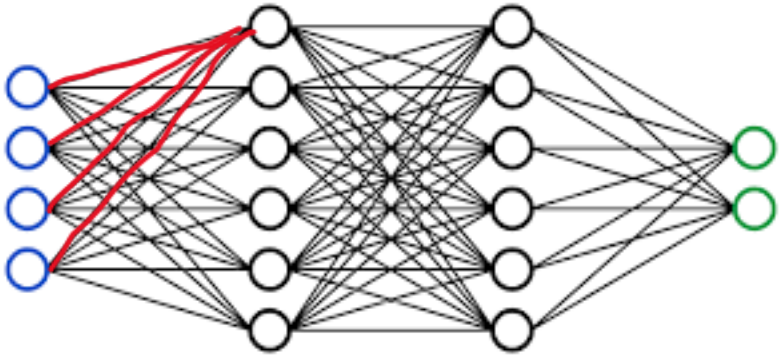
Cannot even solve the XOR problem

Perceptron Limitation (Cont'd)

I. Perceptron is linear model, therefore only works for linearly separable problem.
   - We we can do?
     - Use more perceptron -> Neural Network (NN)
     - Kernel trick – allow us to solve nonlinear problems with linear classifier.

$x = [x_1, x_2, x_3]$



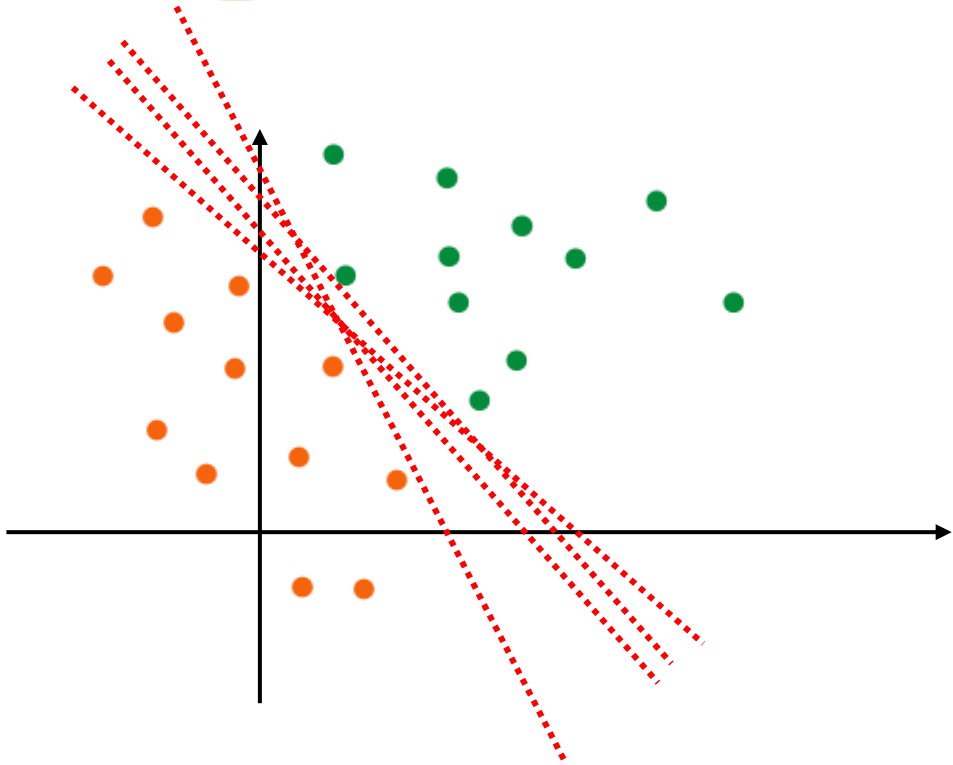


Other Limitations

II. No probability output

III. Sensitive to feature scaling

IV. No margin maximization



In essence, the perceptron is one of the earliest and simplest linear classifiers and forms the foundation of modern neural networks.

After-Class Questions (Concepts)

- Why does the Perceptron update its weights only when a sample is misclassified?
- Explain why the Perceptron does not provide probabilistic outputs.
- What happens if the data is not linearly separable?
- Why does Perceptron not necessarily find the maximum margin solution?
- Explain why the Perceptron loss is convex but non-smooth.

After-Class Questions (hands-on)

- Implement the Perceptron from scratch (without sklearn).
- Modify your implementation to:
  - Track number of updates
  - Plot training error over epochs
- Compare **Perceptron** and **Logistic Regression** on the same dataset. What differences do you observe?